Lic. em Eng. de Sistemas Informáticos

Sistemas Operativos

2025/2026

Enunciado do Trabalho Prático

Este trabalho prático tem como objetivo a aplicação prática dos conceitos de gestão de processos, de ficheiros e de containers. O trabalho visa a construção de um mini-sistema de gestão de ficheiros com um interpretador de comandos personalizado.

O trabalho deverá ser realizado em grupos de 3 elementos, excepcionalmente com outro número de elementos, devidamente justificado e aceite pelo docente.

Parte 1) Implementação um conjunto de comandos para manipulação de ficheiros (10 vals)

Implemente os seguintes comandos através de funções de chamada ao sistema (*system calls*) (ver https://linux.die.net/man/). Estes comandos devem ser implementados em C (para Linux) e devem correr independemente. Além disso, serão invocados através de um interpretador de comandos. Qualquer mensagem de erro deverá ser apresentada no descritor *stderr*, não utilize programação *shell*, funções LibC (*fopen, fread, etc…*) ou comandos pré-existentes para executar a funcionalidade pretendida.

a) **mostra** ficheiro – Este comando deve apresentar no ecrã **todo** o conteúdo do ficheiro indicado como parâmetro. Caso o ficheiro não exista, o comando deve avisar o utilizador que o ficheiro não existe;

b) **copia** origem destino – Este comando deve criar uma cópia do ficheiro "origem" com o nome "destino". Caso o ficheiro de origem não exista, deve ser apresentado um aviso ao utilizador. Caso o ficheiro de destino já exista, deve ser indicado ao utilizador que o ficheiro destino já existe;

c) **acrescenta** origem destino – Este comando deve acrescentar (todo) o conteúdo da “origem” no final do “destino”. Caso algum dos ficheiros não exista, deve ser apresentado um aviso ao utilizador;

d) **conta** ficheiro – Este comando deve contar e apresentar o número de linhas, palavras e caracteres existentes num ficheiro (semelhante ao wc). Se o ficheiro não existir, deverá ser apresentada uma mensagem de erro ao utilizador;

e) **apaga** ficheiro – Este comando deve apagar o ficheiro com o nome indicado. No caso de o ficheiro indicado **não** existir, e apenas, deve ser apresentado um aviso ao utilizador;

f) **informa** ficheiro – Este comando apresenta a informação do sistema de ficheiros em relação ao ficheiro indicado: tipo de ficheiro (normal, diretoria, link, **etc.**), i-node, tamanho em bytes, utilizador dono em formato textual, permissões em formato simbólico (ex: rwxr-xr--) e datas de criação, leitura e modificação em formato textual;

g) **lista** [directoria] – Este comando deve apresentar uma lista de todas as pastas e ficheiros existentes na directoria indicada ou na diretoria atual se não especificada. Adicionalmente, deve distinguir ficheiros simples de diretorias através de uma indicação textual.

h) **procura** ficheiro padrão – Este comando deve procurar e apresentar todas as linhas do ficheiro que contenham o padrão indicado (comparação de substrings). Cada linha encontrada deve ser apresentada com o respetivo número de linha. Caso o ficheiro não exista ou nenhuma linha seja encontrada, deve ser apresentada uma mensagem adequada ao utilizador;

Parte 2) Implementação de um interpretador de linha de comandos (5 vals)

No sentido de substituir o interpretador de comandos habitual (Bash shell) por um novo interpretador personalizado, deve ser implementada uma aplicação, de nome interpretador, cuja função será a leitura de uma sequência de caracteres da consola e em seguida executar essa sequência como um comando e respetivos argumentos no sistema.

O programa deve mostrar o símbolo "%" como indicação de que está pronto para ler um novo comando do utilizador. O programa deve executar o comando através de primitivas de gestão de processos (fork/exec/wait), sem fazer uso da função system(3). Cada comando escrito deve dar origem a um novo processo. O interpretador deve indicar sempre se o comando concluiu com ou sem sucesso, através do seu código de terminação. No caso de ser utilizado um pipe, o resultado mostrado deverá ser o do último comando.

O programa deve permitir executar vários comandos sequencialmente até o utilizador escrever o comando especial "termina" que termina a aplicação.

Funcionalidades adicionais do interpretador

Estas funcionalidades adicionais são exclusivas, isto é, o interpretador pode assumir que o comando inserido apenas faz uso de apenas uma delas, de cada vez.

Redirecionamento de saída (>) – O interpretador deve suportar o redirecionamento da saída padrão (stdout) de um comando para um ficheiro. Exemplo: "lista /home > saida.txt" deve gravar o resultado do comando lista no ficheiro saida.txt. Deve ser implementado usando open() e dup2();

Redirecionamento de entrada (<) – O interpretador deve suportar o redirecionamento da entrada padrão (stdin) a partir de um ficheiro. Exemplo: "conta < dados.txt" deve ler o conteúdo a partir do ficheiro dados.txt. Deve ser implementado usando open() e dup2();

Pipes (|) – O interpretador deve suportar a ligação da saída de um comando à entrada de outro usando pipes. Exemplo: "lista /tmp | conta" deve encaminhar o resultado do comando lista para o comando conta. Deve ser implementado usando pipe(), fork() e dup2(). É suficiente suportar um único pipe entre dois comandos;

Exemplo de utilização:

$ ./interpretador

% lista /home/user/Desktop

...

Terminou comando lista com código 0

% mostra ficheiro.txt > copia.txt

Terminou comando mostra com código 0

% lista /tmp | procura .log

...

Terminou pipe com código 0

% termina

$

Parte 3) Utilização Containers (3 vals)

Crie um Dockerfile que permita construir uma imagem Docker contendo o interpretador e todos os comandos desenvolvidos nas partes anteriores. A imagem deve:

- Partir de uma imagem base adequada (ex: ubuntu, alpine ou fedora) e compilar o código fonte durante a construção da imagem ;
- Definir o interpretador como o comando de entrada (ENTRYPOINT) do container;

Documente os comandos Docker necessários para construir a imagem e executar o container. Demonstre o funcionamento do interpretador dentro do container.

Parte 4) Análise ao Sistemas de Ficheiros (2 vals: 1+1)

Com base no ficheiro *fs.img* (disponível no moodle), responda às seguintes questões:

a) Identifique o tipo de sistema de ficheiros, o tamanho do bloco e o número total de inodes
b) Indique o conteúdo do ficheiro com o nome ipca.txt e dos restantes ficheiros presentes no sistema de ficheiros

Em todos os passos deve descrever os passos necessários, completando a sua resposta com os *screenshots* necessários.

Listagem de Chamadas ao Sistema

As chamadas ao sistema que poderão ser utilizadas neste trabalho encontram-se na secção 2 das páginas do Manual de Unix[1].

Gestão de Ficheiros

- open
- read
- write
- close
- dup e dup2
- pipe
- stat(2)
- opendir(3)
- readdir(3)
- closedir(3)

Gestão de Processos

- fork
- execve(2)
- exec(3)
- wait

As seguintes funções **não** devem ser usadas na resolução deste trabalho:

- system, fopen, fclose, fread, fwrite, fseek

[1] https://linux.die.net/man/2/

Entrega Final

O grupo deverá entregar um arquivo **ZIP**, via Moodle, com a identificação dos membros do grupo e com os ficheiros solicitados. Dentro do arquivo deverão estar os seguintes ficheiros:

- relatório (PDF) documentando a resposta a cada alínea deste enunciado;
- código fonte usado para responder às partes do enunciado;
- Dockerfile desenvolvido na Parte 3;
- não colocar ficheiros executáveis.